

Automation Through Programming

Student Edition — VEX EXP · C++ in VS Code

Name: _____ Date: _____ Period: _____

Introduction

Time to make a robot run **on its own**. You'll add the two pieces that turn a machine into an automatic one: **reading a sensor** for input, and using **if / else** to decide from what it reads. Wrap that in a **while loop** and the robot senses, decides, and acts over and over, with no driver — the **sense → decide → act** loop from Chapter 8, in code. Then you'll **design, build, and program a real automatic device** as a team. Read Chapter 11 in the textbook for the full explanation.

Learning Objectives

- Explain open- vs. closed-loop systems and identify automation as a closed loop (sense → decide → act).
- Read a sensor in code (the Distance Sensor's `objectDistance`, the Bumper Switch's `pressing()`) and use it as a condition.
- Write if / else and a while (true) loop so a motor responds to a sensor.
- Design, build, and program an automatic device from the kit, using the engineering design process and teamwork.
- Store a sensor reading in a variable, read a Potentiometer V2's angle and calibrate it against a captured baseline, and use a for loop to average readings.

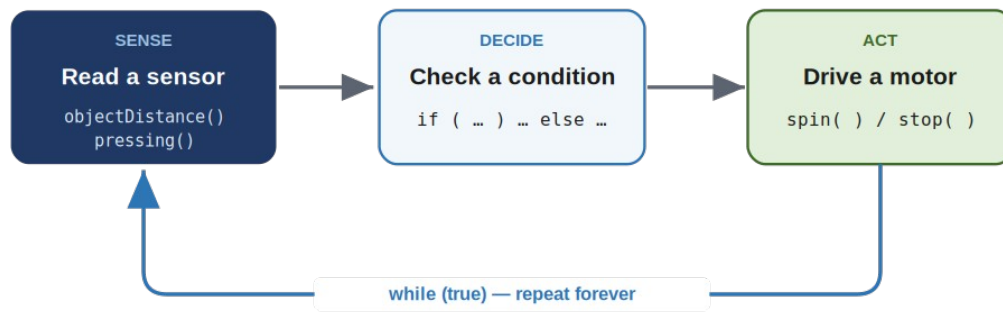
Key Ideas (quick reference)

- **Automation is a closed loop:** sense (read a sensor) → decide (if / else) → act (move a motor), repeated with a while (true) loop. It is the Chapter 8 feedback loop, written in code.
- **Reading a sensor:** name it in your device list, then ask for its reading. `distance.objectDistance(mm)` returns a number like 340 (Smart Port); `bumper.pressing()` returns true or false (3-Wire port). A reading is just a value your program can check.
- **Deciding with if / else:** a condition (a true/false question) goes in the parentheses; one block runs if true, another (with else) if false. Use comparisons `<`, `>`, `==`; chain else if for more than two cases; curly braces mark each block.
- **Repeating with while (true):** an if checks once; a real device must check continuously, so while (true) repeats the block forever. Always put `wait(20, msec)` at the end of the loop, and make sure something inside it can change.
- **Variables, the potentiometer & calibration:** a variable is a named box that stores a value (int, double, bool). The Potentiometer V2 reads a shaft angle (0 to ~330°) on a 3-Wire port, but its zero depends on mounting — so calibrate: capture the home reading into a baseline variable, then use `raw - baseline`. A for loop repeats a set number of times (e.g., average 20 readings for a steady baseline); while (true) repeats forever.

The Sense–Decide–Act Loop

THE SENSE – DECIDE – ACT LOOP

a robot that senses and responds on its own = automation (a closed loop)



This is the feedback loop from Chapter 8 — now written in code.

Read a sensor, decide with if/else, move a motor — and a while loop repeats it forever so the robot keeps responding. This is the Chapter 8 feedback loop, in code.

The Automation Pattern

Name a sensor in your device list, just like a motor:

Two sensors in the device list

```
// name a sensor just like a motor:
motor driveMotor = motor(PORT1);
distance frontEye = distance(PORT2);           // Smart Port
bumper startButton = bumper(Brain.ThreeWirePort.A); // 3-Wire port
```

Then sense, decide, and act inside a forever loop. This program drives forward but stops itself before hitting something:

A complete automatic behavior

```
int main() {
    vexcodeInit();
    driveMotor.setVelocity(50, percent);

    while (true) {
        if (frontEye.objectDistance(mm) < 150) { // keep checking, forever // something is close
            driveMotor.stop();                  // -> stop
        } else {
            driveMotor.spin(forward);            // -> keep driving
        }
        wait(20, msec);                         // pause, then check again
    }
}
```

Read it as the diagram: **sense** (read `frontEye`), **decide** (the `if`), **act** (stop or spin), then loop back. The condition is the part in parentheses — a comparison like `< 150`, or a yes/no reading like `startButton.pressing()`. Swap the sensor and you get a different machine.

Reading an Angle: Variables, the Potentiometer & Calibration

The bumper gives a yes/no and the Distance Sensor gives millimeters. To know *how far a shaft has turned* — an arm, a dial — you need a new sensor and one new tool: a place to **store a value**.

Variables. A variable is a named box that holds a value you can read, use in math, and change. Give it a type and a name; update it (`count = count + 1`) or read it in an `if`. Variables let you save a reading and reuse it, keep a running total, or remember something between loop passes.

Three variables

```
int    count    = 0;           // a whole number
double home     = 0;           // a number w/ decimals
bool   running  = false;       // true or false
```

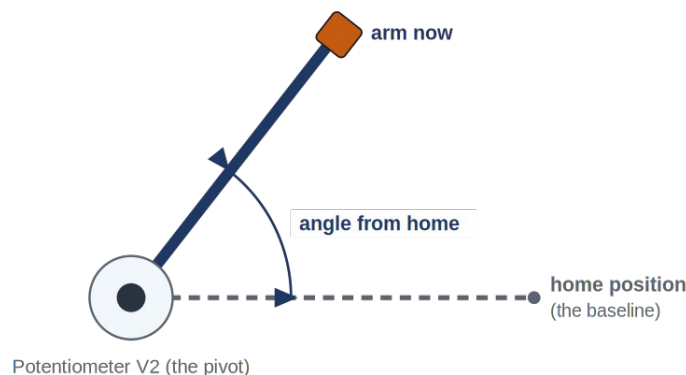
The Potentiometer V2. It measures the angle of the shaft it is mounted on, 0 to about 330°, through a lettered 3-Wire port. You read it as a decimal angle, so store it in a double. Unlike the bumper (pressed or not) or the Distance Sensor (mm to an object), it reports a rotation — perfect for an arm or dial — and it keeps its reading after a restart because the value follows the shaft.

Name it in the device list, then read it

```
// in the device list, above main:
potV2 armPot = potV2(Brain.ThreeWirePort.A);

// later, read the angle into a double:
double raw = armPot.angle(degrees); // 0 to ~330
```

Calibration — where is zero? The potentiometer's zero is wherever it was mounted (its slots even let you nudge it 90° by hand), so a raw 145° is 145° from the sensor's own arbitrary zero, not from level. To fix it, calibrate: at startup, hold the arm at a known home, read the sensor, and store that reading in a variable — the baseline. Then $\text{raw} - \text{baseline}$ is the angle from home. Same idea as the inertial's calibrate step (Ch13); here you do it yourself, with a variable.



RULE: capture the raw reading once at home (the **baseline**), then $\text{angle from home} = \text{raw} - \text{baseline}$.

Capture the raw reading once at the home position (the baseline); then the arm's angle from home = raw - baseline. The sensor's own zero is wherever it was mounted.

The for loop. One analog reading jitters, so average several for a steady baseline — which needs a loop that runs a fixed number of times: a for loop. It starts a counter, runs while it is below a limit, and steps it each pass. Now you know two kinds of loop: while (true) repeats forever; a for loop repeats a set number of times, then moves on.

Calibrate: average 20 readings at home

```
double sum = 0;
for (int i = 0; i < 20; i = i + 1) {
    sum = sum + armPot.angle(degrees);
    wait(5, msec);
}
double home = sum / 20;           // the baseline
```

Putting it together. Calibrate once at the top of main(), then read the calibrated angle in the while (true) loop and act — raise an arm until it passes 90° from home, then hold. A classic challenge:

hold the arm at three heights, one per controller button. Always subtract the baseline before trusting a potentiometer number, and re-calibrate after remounting.

Raise the arm to 90° from home, then hold

```
int main() {
    vexcodeInit();

    // calibrate: average 20 readings at home
    double sum = 0;
    for (int i = 0; i < 20; i = i + 1) {
        sum = sum + armPot.angle(degrees);
        wait(5, msec);
    }
    double home = sum / 20;

    armMotor.setStopping(hold);
    while (true) {
        double angle = armPot.angle(degrees) - home;
        if (angle < 90) {           // not high enough yet
            armMotor.spin(forward);
        } else {
            armMotor.stop();
        }
        wait(20, msec);
    }
}
```

Build Constraints

Build constraints (still in force). (1) Build your device from the classroom PLTW Gateway kit only — its bundled motor and sensors are fair game; the separate competition team's V5 parts are off-limits. (2) The whole device must fit one IKEA Kallax cubby — a 33 × 33 cm (13 × 13 in) opening, about 38 cm (15 in) deep. The 33 × 33 face is the binding limit, so plan your gate, sign, or elevator to pack into that square.

Safety

- An automatic robot moves on its own, sometimes when you don't expect it. Keep hands clear of gates, arms, and gears, and be ready to power down.
- Secure the device so a moving part can't pinch fingers or knock things off the bench.
- Run a program that drives only after you've unplugged the USB-C cable.
- Power the Brain down with the X button when you are done, and store the device in its cubby.

Equipment & Materials

- Your computer with VS Code and the VEX extension, and a USB-C cable
- The PLTW Gateway EXP kit: Brain, charged battery, at least one motor, and a sensor (Distance, Bumper, Optical, or Potentiometer V2)
- Your engineering notebook for sketches, pseudocode, and a daily journal
- This project worksheet (design brief, decision matrix, and reflection)

Procedure

1. **Define the problem.** Read your task, then fill in the design brief (Worksheet 1): what it must do and every constraint — kit only, fits the cubby, which sensor.
2. **Sketch and choose.** Each teammate sketches a design. Compare them in the decision matrix (Worksheet 2) against your criteria, and pick the best one to build.
3. **Build the mechanism.** Construct your device from the kit so it fits the cubby. Mount the motor and the sensor where they belong, and wire them to the Brain.
4. **Plan, then code.** Write pseudocode for the sense → decide → act loop (Worksheet 3) first. Then translate it to C++: name your devices, set up the while (true) loop, and fill in the if / else.
5. **Test and refine — a little at a time.** Download, watch it run, and fix one thing at a time. Tune your distance threshold or speed until it behaves the way the task needs.
6. **Evaluate.** Judge how well it works, keep your daily journal (Worksheet 4), and answer the conclusion questions — including whether your device is an open or a closed loop.

Worksheet 1 — Design Brief

Read your task and define the problem before you build.

Section	Your answer
Client / who needs it	
Problem statement — what is the need?	
What it must do (design statement)	
Constraints (kit only, fits cubby, sensor)	
Sensor(s) you'll use	

Worksheet 2 — Decision Matrix

List your criteria across the top, then score each idea 1–4 (1 = barely meets it, 4 = best possible). Add across for each idea's Total — the highest is your best solution.

Idea	Criteria 1	Criteria 2	Criteria 3	Total
A				
B				
C				
D				

Worksheet 3 — Plan the Code

Match your project to a pattern (A–D) from the chapter, then fill in the sense → decide → act loop in plain language **before** you write C++. To pick your threshold, read the sensor value on the Brain's Devices screen at the real trigger distance.

Pseudocode

```
/*
while (true)
{
    read the sensor: _____

    if ( _____ )
    {
        _____
    }
    else
    {
        _____
    }
}
*/
```

Worksheet 4 — Daily Journal

A couple of sentences per day — especially what got in your way and how you fixed it.

Day	What we did / obstacles we hit and how we solved them
1	
2	
3	

How You'll Be Assessed

- It works as an **automatic, closed-loop** device — senses and responds with no one driving it.
- The code is a clear **sense** → **decide** → **act** loop — a sensor read, an if/else, a motor — inside while (true) with a wait.
- The **design process is documented** — brief, decision matrix, pseudocode, and a daily journal.
- You can **point to the exact line** that senses, the line that decides, and the line that acts in your own program.

Conclusion Questions

Answer in complete sentences.

1. What sensor(s) did you use to solve this problem, and why those?
2. Was your device an open-loop or a closed-loop system? Why?
3. Describe two malfunctions your team had to troubleshoot, and how you overcame them.
4. Describe two responsibilities you held during this project.

5. Why does an automatic device need a while (true) loop instead of a single if?
6. Write the sense–decide–act idea as pseudocode for this device: "a night-light motor turns a lamp on when the room is dark." Name the sensor reading, the decision, and the action.
7. A Potentiometer V2 on a robot arm reads 160 when the arm is parked at its home position. Why can't you treat that as "160°"? What do you store in a variable to fix it, and how do you then find the arm's angle from home? Where would a for loop help?